

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

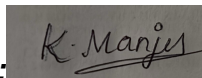
5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I am K.Manjunadha Reddy / 192372081 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Answers

1. XML Parsing Algorithm

ALGORITHM ParseXMLDocument

BEGIN

STEP 1: Start the program

STEP 2: Load the XML document

xmlDocument ← LoadXML("users.xml")

STEP 3: Check whether the XML file was loaded successfully

IF xmlDocument = NULL THEN

Display "Error: Unable to load XML document"

STOP

END IF

STEP 4: Access the root element

root ← GetRootElement(xmlDocument)

STEP 5: Traverse each child node under the root

FOR EACH user IN root.children DO

STEP 6: Check whether the current node is a user element

IF user.name = "user" THEN

STEP 7: Extract user details

id ← GetAttribute(user, "id")

name ← GetElementValue(user, "name")

email ← GetElementValue(user, "email")

phone ← GetElementValue(user, "phone")

STEP 8: Display the extracted information

Display "-----"

Display "User ID : " + id

Display "Name : " + name

Display "Email : " + email

Display "Phone : " + phone

```

STEP 9: Traverse nested elements if present
FOR EACH child IN user.children DO
    IF child.name = "address" THEN
        city ← GetElementValue(child, "city")
        state ← GetElementValue(child, "state")

        Display "City  : " + city
        Display "State : " + state
    END IF
END FOR

END IF

END FOR

STEP 10: Close the XML document

STEP 11: End the program

END

```

Flow

```

Load XML
↓
Access Root Node
↓
Traverse User Nodes
↓
Extract Attributes
↓
Extract Child Elements
↓
Traverse Nested Elements
↓
Display Data
↓
End

```

2. MVC Workflow

ALGORITHM MVC_WebApplication

BEGIN

STEP 1: Start the web application

STEP 2: User sends a request
request ← ReceiveRequest()

STEP 3: Web server receives the request

STEP 4: Identify the requested URL and HTTP method
url ← GetURL(request)
method ← GetMethod(request)

STEP 5: Send the request to the appropriate Controller
controller ← FindController(url)

STEP 6: Controller receives the request

STEP 7: Controller validates the request
IF request is invalid THEN
 Display error response
 STOP
END IF

STEP 8: Controller calls the appropriate Model
model ← CreateModel()

STEP 9: Model establishes connection with the database

STEP 10: Model executes the required database operation
data ← Model.GetData()

STEP 11: Database processes the query

STEP 12: Database returns the result to the Model

STEP 13: Model processes the returned data

STEP 14: Model sends the processed data to the Controller

STEP 15: Controller sends data to the View

STEP 16: View receives the data

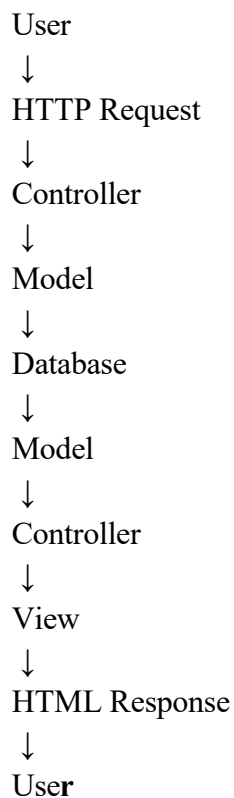
STEP 17: View generates the HTML response

STEP 18: Web server sends the generated response to the client

STEP 19: Browser displays the response to the user

END

MVC Flow



3. JSP Request Flow

ALGORITHM JSP_RequestResponse

BEGIN

STEP 1: Start the web application

STEP 2: Client requests a JSP page
request ← ReceiveRequest()

STEP 3: Web server receives the request

STEP 4: Identify the requested JSP file

jspPage ← FindJSP(request)

STEP 5: Check whether the JSP has already been compiled

IF jspPage is not compiled THEN

Translate JSP into Servlet

Compile the generated Servlet

END IF

STEP 6: Execute the JSP Servlet

STEP 7: Retrieve request parameters

username ← GetParameter(request, "username")

STEP 8: Validate the received parameters

IF parameters are invalid THEN

Generate error message

ELSE

STEP 9: Call backend component

backendData ← CallServletOrService(username)

STEP 10: Backend component interacts with database

data ← ExecuteDatabaseOperation(username)

STEP 11: Database returns the required data

STEP 12: Backend sends data back to JSP

STEP 13: JSP combines static HTML with dynamic data

END IF

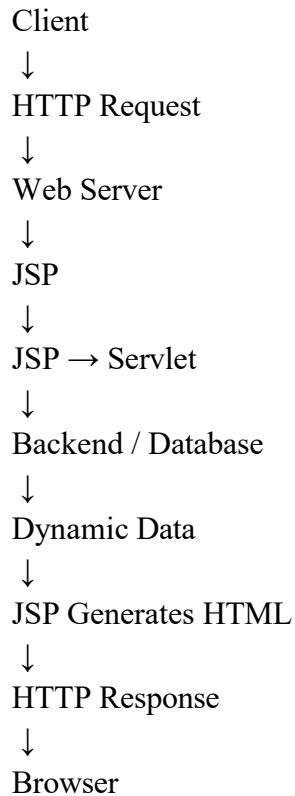
STEP 14: Generate HTML response

STEP 15: Send HTML response to the client

STEP 16: Browser receives and displays the webpage

END

JSP Flow



4. Database Retrieval Algorithm

ALGORITHM RetrieveDatabaseData

BEGIN

STEP 1: Start the program

STEP 2: Load the database driver

STEP 3: Define database connection details

```
databaseURL ← "Database URL"
username ← "Database Username"
password ← "Database Password"
```

STEP 4: Establish database connection

```
connection ← Connect(databaseURL, username, password)
```

STEP 5: Check whether connection was successful


```
IF connection = NULL THEN
    Display "Database connection failed"
    STOP
END IF
```

STEP 6: Create SQL query

```
query ← "SELECT id, name, email FROM users"
```

STEP 7: Create a statement or prepared statement

```
statement ← CreateStatement(connection)
```

STEP 8: Execute the SQL query

```
resultSet ← ExecuteQuery(statement, query)
```

STEP 9: Check whether records are available

```
IF resultSet is empty THEN
    Display "No records found"
ELSE
```

STEP 10: Process each record

```
WHILE resultSet has next record DO
```

```
    id ← GetValue(resultSet, "id")
    name ← GetValue(resultSet, "name")
    email ← GetValue(resultSet, "email")
```

STEP 11: Display retrieved data

```
    Display "ID   : " + id
    Display "Name : " + name
    Display "Email : " + email
    Display "-----"
```

```
END WHILE
```

```
END IF
```

STEP 12: Close the result set

STEP 13: Close the statement

STEP 14: Close the database connection

STEP 15: End the program

END

Database Flow

Start



Load Driver



Connect to Database



Create SQL Query



Execute Query



Receive Result Set



Process Records



Display Data



Close Resources



End

5. PHP Validation Algorithm

ALGORITHM ValidatePHPForm

BEGIN

STEP 1: Start the program

STEP 2: Receive form data

name ← GetInput("name")

email ← GetInput("email")

password ← GetInput("password")

STEP 3: Create an empty error list

errors ← EmptyList()

STEP 4: Check whether the name is empty

IF name = EMPTY THEN

Add "Name is required" to errors

END IF

STEP 5: Check whether the email is empty

IF email = EMPTY THEN

 Add "Email is required" to errors

ELSE

 STEP 6: Validate email format

 IF EmailFormatIsValid(email) THEN

 Add "Enter a valid email address" to errors

 END IF

END IF

STEP 7: Check whether the password is empty

IF password = EMPTY THEN

 Add "Password is required" to errors

ELSE

 STEP 8: Check password length

 IF Length(password) < 8 THEN

 Add "Password must contain at least 8 characters" to errors

 END IF

END IF

STEP 9: Check the error list

IF errors is NOT EMPTY THEN

 STEP 10: Display all validation errors

 FOR EACH error IN errors DO

 Display error

 END FOR

 STEP 11: Reject form submission

ELSE

 STEP 12: Sanitize and process valid input

 name ← Sanitize(name)

 email ← Sanitize(email)

 STEP 13: Allow form submission

STEP 14: Store/process the valid data

Display "Form submitted successfully"

END IF

STEP 15: End the program

END

PHP Validation Flow

User Enters Data



Receive Form Data



Check Empty Fields



Validate Email



Validate Password



Any Errors?



YES



NO



Show Errors



Sanitize Data



Reject



Process/Store



Success Response